IIoT and Cloud Computing (CS5005)

Complete Study Notes

Gyan Chandra, CS23I1053, IIIT Kancheepuram

AY 2025–26

Contents

1 Lecture 1: Course Overview **5**
1.1 Course Objectives . . . 5
1.2 Syllabus Topics . . . 5
1.3 Reference Books . . . 5
1.4 Evaluation Scheme . . . 5

2 Lecture 2: History & Origin of IoT **6**
2.1 Timeline of IoT Evolution . . . 6
2.2 Internet of Everything (IoE) – Cisco . . . 6
2.3 IoT vs IIoT and Global Players . . . 7
2.4 Market Statistics (2025 Forecast) . . . 7
2.5 Definitions of IoT by Global Giants . . . 7

3 Lecture 3: IoT Stack Architecture **7**
3.1 Computing Models . . . 7
3.1.1 1. Edge Computing (Onboard Computing) . . . 8
3.1.2 2. Cloud Computing . . . 8
3.1.3 3. Fog Computing . . . 8
3.2 IoT Reference Model by IoT World Forum (2014) . . . 8
3.3 IoT Reference Model by IBM (2020) . . . 10
3.4 IoT Data Flow Summary . . . 10

4 Lecture 4: Sensing Technologies **10**
4.1 What is a Sensor? . . . 10
4.1.1 Temperature Sensor . . . 10
4.1.2 Gas/Chemical Sensor . . . 10
4.1.3 Optical Sensor . . . 10
4.1.4 Proximity Sensor (Ultrasonic) . . . 11
4.2 Sensor Characteristics (Important) . . . 11
4.3 Common IoT Sensors . . . 12
4.4 Passive Infrared Sensor (PIR) . . . 12
4.5 Classification of Sensors . . . 12

4.5.1 1. Scalar vs Vector . . . 12
4.5.2 2. Analog vs Digital . . . 12
4.5.3 3. Active vs Passive . . . 13
4.6 Actuators . . . 13

5 Lecture 5: Single Board Computers (SBC) 13
5.1 Smart Objects . . . 13
5.1.1 Five Components of a Smart Object . . . 13
5.2 Arduino Boards . . . 14
5.2.1 Pin Functions . . . 14
5.3 ESP Family Boards . . . 14
5.4 Raspberry Pi . . . 14
5.5 Other SBCs . . . 15

6 Lecture 7N, 10 & 12: Communication in IoT 15
6.1 Communications Criteria & Range . . . 15
6.2 IoT Network Topologies . . . 15
6.3 Frequency Bands . . . 16
6.4 Constrained Devices (RFC 7228) . . . 16
5.5 Wi-Fi & Li-Fi . . . 16
5.6 Bluetooth . . . 16
5.7 LPWAN (Low Power Wide Area Networks) . . . 17
5.7.1 1. LoRaWAN . . . 17
5.7.2 2. DASH7 . . . 17
5.7.3 3. Sigfox . . . 17
5.7.4 4. NB-IoT (NarrowBand IoT) . . . 17
5.8 RFID, NFC & Zigbee . . . 18
5.8.1 RFID . . . 18
5.8.2 NFC (Near Field Communication) . . . 18
5.8.3 Zigbee (IEEE 802.15.4) . . . 18
5.9 SBC Wiring Protocols (UART, I2C, SPI) . . . 18

6 Lecture 13: Network Layer & 6LoWPAN 19
6.1 IP Adaptation (6LoWPAN) . . . 19

7 Lecture 18 & 19: Application Protocols for IoT 20
7.1 Transport Layer Basics: TCP vs UDP . . . 20
7.2 Why HTTP is NOT suitable for IoT . . . 20
7.3 CoAP (Constrained Application Protocol) . . . 20
7.3.1 The 4 Message Types of CoAP . . . 20
7.3.2 CoAP Header/Message Format . . . 21
7.3.3 Message Exchange Models (Responses) . . . 21
7.3.4 Methods and Response Codes . . . 21
6.4 MQTT (Message Queuing Telemetry Transport) . . . 21
6.4.1 Architecture . . . 22
6.4.2 Topics and Wildcards . . . 22
6.4.3 Quality of Service (QoS) Levels . . . 22
6.4.4 MQTT PUBLISH Packet Structure . . . 23

7 Lecture 21–34: Cloud Computing & Virtualization 23
7.1 HPC vs HTC . . . 23
7.2 Cloud Service Models (Pizza as a Service) . . . 23
7.3 Virtualization . . . 24
7.4 AWS Storage Computing . . . 24
7.5 Data Centers & CAP Theorem . . . 24
6.6 MQTT Control Packets in Detail . . . 25
6.6.1 1. Connection Packets . . . 25
6.6.2 2. Publishing Packets (QoS Mechanisms) . . . 25
6.6.3 3. Subscription Packets . . . 25
6.7 Deep Dive: The CONNECT Packet . . . 26
6.8 MQTT vs HTTP (Crucial Comparison) . . . 26
6.9 MQTT Return Codes . . . 27
6.9.1 CONNACK Return Codes . . . 27
6.9.2 SUBACK Return Codes . . . 27
6.10 CoAP vs MQTT (Crucial Comparison) . . . 27

7 Lecture 21–33: Cloud Computing & Data Centers 28
7.1 History of Computing . . . 28
7.2 Cloud Computing as a Utility . . . 28
7.3 HPC vs HTC . . . 28
7.4 Cloud Service Models (Pizza as a Service) . . . 28
7.5 Data Centers & Distributed Systems . . . 28
7.6 CAP Theorem . . . 29
7.7 Disadvantages of Cloud Computing . . . 29

8 Lecture Series: Cloud Computing In-Depth 29
8.1 Evolution of Computing . . . 29
8.2 HPC vs HTC . . . 30
8.3 Computing Architectures . . . 30
8.4 National Supercomputing Mission (NSM) . . . 30
8.5 The "Utility" Vision & NIST Definition . . . 30
7.6 Cloud Service and Deployment Models . . . 31
7.6.1 Service Models (XaaS) . . . 31
7.6.2 Deployment Models . . . 31
7.7 Shared Responsibility Model . . . 31
7.8 Elasticity vs. Scalability . . . 32
7.9 Virtualization Containerization . . . 32
7.9.1 Hypervisors . . . 32
7.9.2 Full vs. Para-Virtualization . . . 32
7.9.3 Containers vs. VMs . . . 32
7.10 Major Cloud Providers & AWS Ecosystem . . . 33
7.10.1 Key AWS Services . . . 33
7.11 Data Center Architecture & Networking . . . 33
7.11.1 Structural Hierarchy . . . 33
7.11.2 Cooling Systems (Raised Floors) . . . 33
7.11.3 Interconnection Networks & Fat-Tree Topology . . . 34
7.11.4 Modular Data Centers (e.g., SGI ICE Cube) . . . 34
7.12 Cloud Storage Models . . . 34

7.12.1 1. Block Storage (e.g., Amazon EBS) 34
7.12.2 2. File Storage (e.g., Amazon EFS) 34
7.12.3 3. Object Storage (e.g., Amazon S3) 35
7.13 CAP Theorem in Cloud Databases 35
7.14 Service Level Agreement (SLA) 35
7.14.1 Types of SLAs 36

9 Real-World IIoT System Calculations & Engineering 36
9.1 PIR Sensor Battery Life Calculation 36
9.2 Sensor Coverage Area Calculation 36
9.3 Wireless Signal Loss: Friis Equation & FSPL 37
9.3.1 Friis Transmission Equation 37
9.3.2 Free Space Path Loss (FSPL) in dB 37
9.3.3 FSPL Numericals 38
8.4 Networking Protocols & LoRa Planning 38
8.4.1 The Hidden Terminal Problem & CSMA/CA 38
8.4.2 LoRa Network Planning: Fresnel Zone & Earth Bulge 38
8.5 AODV Routing Protocol (Ad hoc On-Demand) 39
8.6 IP Addressing & Wi-Fi Congestion 39
8.7 Cellular Networks & Access Technologies 39

1 Lecture 1: Course Overview

1.1 Course Objectives

1. Learn the concepts of IoT and Cloud Computing.
2. Understand existing architectures and frameworks.
3. Design and implement a prototype IoT/cloud system.

1.2 Syllabus Topics

IoT Enabling Technologies (Physical/Logical Design), IoT Communication, Physical Devices, Cloud Models, Virtualization, Case Studies on Industrial Automation.

1.3 Reference Books

- *Internet of Things: A Hands-On Approach* – Arshdeep Bahga & Vijay Madisetti
- *Cloud Computing: A Hands-On Approach* – Arshdeep Bahga & Vijay Madisetti

1.4 Evaluation Scheme

Component	**Marks**
Mid-Semester Exam (MSE)	25
End-Term Exam (ETE)	50
Quiz / Assignment	25
Total	**100**

Important Rule: MSE + ETE $\geq$ 15 marks is *mandatory*. Even if Assignment = 25/25, failing to score $\geq$ 15 in theory exams means **Fail**. Minimum 85% attendance is compulsory.

2 Lecture 2: History & Origin of IoT

2.1 Timeline of IoT Evolution

Year	Milestone
1969	ARPANET launched (grandfather of internet).
1971	Intel 4004 chip – first microprocessor.
1982	First internet-connected appliance: Coca-Cola vending machine at Carnegie Mellon University (by David Nichols).
1989	World Wide Web (WWW) by Tim Berners-Lee (2016 Turing Award).
1990	John Romkey creates first internet-controlled toaster.
1994	Mark Weiser coins "ubiquitous computing".
1998	MIT Auto-ID Center launches for RFID research.
1999	**Kevin Ashton coins the term "Internet of Things"** for RFID supply chain presentation.
2000	LG announces first internet-connected smart refrigerator.
2007	Smartphones (iPhone) reinvent human-device interfaces.
2008	Number of connected devices **exceeds human population**.
2009	Industrial IoT (IIoT) concept begins.
2012–14	Cisco introduces **Internet of Everything (IoE)**.
2015	Mirai botnet attack exposes IoT security weaknesses.
2024+	Future: AIoT (AI + IoT) and Satellite IoT.

Exam Focus

1. Who coined the term "Internet of Things" and in which year?
   Ans: Kevin Ashton, 1999.
2. Which was the world's first internet-connected appliance?
   Ans: Coca-Cola Vending Machine at Carnegie Mellon University, 1982.
3. In which year did connected devices cross the human population?
   Ans: 2008.

Memory Trick

"99 ke chakkar mein Kevin phas gaya" ⇒ 1999, Kevin Ashton.
"1982 mein pehli baar Internet ne thandi COKE pee."

2.2 Internet of Everything (IoE) – Cisco

Definition

Internet of Everything (IoE): The networked connection of **People, Process, Data,** and **Things**.
– 99.4% of physical objects are *still* unconnected (as of Cisco's report).

Three types of connections in IoE:

- **P2P** – People to People
- **M2P** – Machine to People
- **M2M** – Machine to Machine

Exam Focus

What are the 4 pillars of Cisco's IoE?
Ans: People, Process, Data, Things.

Memory Trick

"P-P-D-T" ⇒ **P**apa **P**adhne **D**o **T**ab-tak ⇒ People, Process, Data, Things.

2.3 IoT vs IIoT and Global Players

- **Consumer IoT:** Home Automation, Appliances, Wearables, Phones.
- **Industrial IoT (IIoT):** Transportation, Logistics, Utilities, Manufacturing.
- **Key Players:** GE Digital, Cisco, Amazon, Bosch, Samsung.

2.4 Market Statistics (2025 Forecast)

- Global active IoT connections: **42.8 billion** (vs. human population of 8.3 billion).
- LPWA network CAGR: **36%**.
- 80% of connections via Wi-Fi, Bluetooth, or LPWA.

2.5 Definitions of IoT by Global Giants

Gartner: "The network of physical objects that contain embedded technology to communicate and sense or interact with their internal states or the external environment."

IBM: Physical objects "embedded with sensors, software and network connectivity that allows them to collect and share data."

AWS: Technology that "facilitates communication between devices and the cloud."

Exam Focus

Define IoT. (Use Gartner or IBM definition for full marks.)

3 Lecture 3: IoT Stack Architecture

3.1 Computing Models

There are three main paradigms for processing IoT data:

3.1.1 1. Edge Computing (Onboard Computing)

Compute directly on the device (sensor) itself.

- **Pros:** Data is available immediately, Data doesn't leave the device (Privacy/Security).
- **Cons:** Limited computing and storage capacity, Power constraints (Battery drains faster).

Example: Face unlock on a smartphone.

3.1.2 2. Cloud Computing

Computing in large Data Centers.

- **Pros:** Full power of cloud computing, Unlimited processing power, Autoscaling, Available 24/7.
- **Cons:** High network bandwidth required, High roundtrip latency (Delay), Vulnerable to internet disconnections.

3.1.3 3. Fog Computing

Computing at one hop wireless distance from the sensors (e.g., at a local router/gateway).

- **Pros:** Brings the cloud closer to the edge, Low roundtrip latency, Low bandwidth consumption, Allows operation in a disconnected environment.

Exam Focus

Compare Edge, Cloud, and Fog Computing with their Pros & Cons.

Memory Trick

"Zameen (Edge) aur Aasman (Cloud) ke beech ka Dhundh/Badaal Fog kehlata hai!"

3.2 IoT Reference Model by IoT World Forum (2014)

Published by the IoT World Forum Architectural Committee (led by Cisco, IBM, etc.). It consists of **Seven Layers**.

Layer 7: Collaboration & Processes (People & Business Processes)

Layer 6: Application (Reporting, Analytics, Control)

Layer 5: Data Abstraction (Aggregation & Access)

Layer 4: Data Accumulation (Storage)

Layer 3: Edge Computing (Data Elements Analysis & Transformation)

Layer 2: Connectivity (Communication & Processing Units)

Layer 1: Physical Devices & Controllers (The "Things")

Detailed Layer Explanation:

1. **Physical Devices & Controllers:** Represents the "things" (Hardware sensors and actuators). Collects raw data like temperature, pressure.
2. **Connectivity (Network Layer):** Facilitates communication using Wi-Fi, Zigbee, etc. Contains Gateways and Backhaul Networks.
3. **Edge Computing:** Data reduction, filtering, aggregation, simple analytics. Assesses if threshold is crossed for alerts (Filters garbage data before cloud).
4. **Data Accumulation:** Responsible for storing and organizing the raw data.
5. **Data Abstraction:** Transforms raw data into meaningful insights and performs data modeling (Tables/Models).
6. **Application:** Enables user interactions through dashboards, apps. Provides reports.
7. **Collaboration & Processes:** Deriving actionable insights and strategies to achieve business goals based on application data.

Exam Focus

Explain the 7-Layer IoT Reference Model by IoT World Forum in detail. (High Probability)

Memory Trick

Read Bottom to Top (Layers 1 to 7): **P-C-E-Da-Da-A-C**
"**P**apa ne **C**onnect kiya **E**dge se, aur **D**ata **D**ata **A**aya **C**loud pe"
(Physical, Connectivity, Edge, Data Accumulation, Data Abstraction, Application, Collaboration)

3.3 IoT Reference Model by IBM (2020)

- **3 Horizontal Layers:** Devices, Edge, Cloud.
- **3 Vertical Layers:** Identity Management, Access Control, Data Security.
- **Core Philosophy:** Implement **"Security by Design"**.

3.4 IoT Data Flow Summary

Hardware Sensor → SPI/I2C Wire → Microcontroller → Wi-Fi/RF → MQTT Message → User Application.

4 Lecture 4: Sensing Technologies

4.1 What is a Sensor?

Definition

A device that detects changes in the physical world (like temperature, light, or motion), then converts that information into an electrical signal, and finally into a digital data format/boolean value.

4.1.1 Temperature Sensor

- Contains a material like a **Thermistor** that physically changes its resistance when it gets hot or cold.
- As temperature rises, it becomes harder (or easier) for electricity to flow.
- This change in resistance alters the current, which is measured and converted to a numerical value.

4.1.2 Gas/Chemical Sensor

- Surface has a chemical layer that reacts with specific gases.
- Reaction creates a tiny flow of current (amperage) proportional to the number of gas molecules.
- Output is converted into **PPM (Parts Per Million)**.

4.1.3 Optical Sensor

- Uses a Photodiode or **LDR (Light Dependent Resistor)**.
- When photons (light particles) hit the surface, they knock electrons loose, creating flow.
- **LDR Principle:** Resistance drops as light increases. Converted into a **"Lux"** value (light intensity).

4.1.4 Proximity Sensor (Ultrasonic)

- Emits a sound wave (ping) and detects the **Time of Flight** (echo bounce time).
- The sensor generates a voltage pulse whose duration (width) corresponds to the time taken.
- Microcontroller calculates the distance.

Exam Focus

1. Define a Sensor. Explain the working principle of a Temperature Sensor.
2. How does an Ultrasonic Proximity Sensor calculate distance?
   Ans: Using Time of Flight of sound waves.

4.2 Sensor Characteristics (Important)

1. **Range:** Gives the highest and lowest value. Beyond this value, there is no sensing.
2. **Accuracy:** How well the sensor measures the environment in an absolute sense (Difference between standard and measured value).
3. **Resolution:** The smallest change in the input that the sensor can detect (e.g., 0.1°C). Higher resolution = better precision.
4. **Repeatability:** Ability to repeat a measurement when put back in the same environment. (For same input, same output).
5. **Drift:** Low frequency change in a sensor with time. Associated with electronic aging of components.
6. **Sensitivity:** Ratio of incremental change in the response with respect to small change in input.
7. **Reliability:** Ability to perform correctly for a long period without failure.
8. **Response Time:** How fast the sensor reacts to a change in input.

Exam Focus

Define any 4 Sensor Characteristics (Explain Accuracy, Resolution, Drift, and Sensitivity).

Memory Trick

"R-R-R A-S-D" ⇒ Range, Resolution, Repeatability, Accuracy, Sensitivity, Drift.

4.3 Common IoT Sensors

- **DHT11:** Measures Temperature and Humidity. Working voltage 3.3V-5V. (Temp error ±2°C, Humidity error ±5%). Cheap and digital.
- **HC-SR04 (Ultrasonic):** Distance measurement (2cm - 400cm). 4 Pins: VCC, Trig, Echo, GND.
- **IR Obstacle Sensor:** Detects object presence. Built-in IR transmitter and receiver.

4.4 Passive Infrared Sensor (PIR)

- Detects human presence (motion detection).
- A living organism emits heat as IR radiation. The sensor detects this change.
- It is **Passive** because it does not emit energy; it only receives.
- **Core Component: Pyroelectric Sensor**.
- **Fresnel Lens:** The white dome cover that focuses incoming IR radiation onto the pyroelectric sensor.

Exam Focus

Explain the working of a PIR sensor. What is the role of the Fresnel Lens and Pyroelectric Sensor?

4.5 Classification of Sensors

4.5.1 1. Scalar vs Vector

- **Scalar:** Measures only **magnitude**. Output is a single value that doesn't change with orientation. (Ex: Light, Temp, Humidity).
- **Vector:** Measures magnitude **AND direction**. Output is typically x, y, z coordinates. Data changes based on positioning. (Ex: Accelerometer, Magnetometer).

4.5.2 2. Analog vs Digital

- **Analog:** Continuous stream of data. Requires ADC. Cheaper but noise has more effect. (Ex: LM35).
- **Digital:** Data in discrete steps (Binary 0/1). Converts physical measurement into a digital code inside the sensor itself. Slightly costlier but reliable. (Ex: DHT11).

4.5.3 3. Active vs Passive

- **Active:** Emit their own energy (light, sound, radar) and measure the reflection. (Ex: Ultrasonic, Radar).
- **Passive:** Do not emit energy. Detect naturally occurring energy. (Ex: IR cameras, microphones, PIR).

Exam Focus

Differentiate between Scalar and Vector, Analog and Digital, Active and Passive sensors with examples.

4.6 Actuators

Definition

Actuator: Part of the IoT system used to perform the **control action**. Mechanical or electro-mechanical. It requires a control signal input and an energy source.

Examples: Relay (switch), Motor, Diode.
System Flow: Real World → Sense (Sensors) → Digital Representation → Processor → Control Signal → Actuators → Real World.

5 Lecture 5: Single Board Computers (SBC)

5.1 Smart Objects

Definition

Smart Objects: The building blocks of IoT. Transforms everyday objects into a network of intelligent objects. Able to learn from and interact with their environment. (Also known as: Smart sensor, Smart device, IoT device, Intelligent node/thing).

5.1.1 Five Components of a Smart Object

A smart object is essentially a tiny, low-cost computer embedded into an object.

1. **Sensor:** Measures physical data from the environment.
2. **Actuator:** Performs a physical task based on input.
3. **Communication Device:** Receives instructions and routes data to the network.
4. **Power Source:** Provides energy (battery, solar, mains).
5. **CPU & Memory:** For data handling and local processing.

Exam Focus

What are the five core components of a Smart Object?

5.2 Arduino Boards

A popular, open-source single-circuit microcontroller platform.

Board	Digital Pins	Analog Pins	Memory
Arduino Uno	14	6	32 KB
Arduino Mega 2560	54	16	256 KB
Arduino Nano	Compact version of Uno		

5.2.1 Pin Functions

- **Digital Pins:** Provide binary output/input (`HIGH` or `LOW` state).
- **Analog Pins:** Read a continuous range of analog voltages (0V to 5V).
- **PWM Pins (Pulse Width Modulation):** Digital pins that produce a pulsing signal with varying duty cycles. Used to simulate an analog output using digital signals (e.g., LED dimming, motor speed control).

Exam Focus

What is the use of PWM pins in Arduino?

5.3 ESP Family Boards

Unlike Arduino, ESP boards come with built-in wireless capabilities, making them highly suitable for IoT.

- **ESP32:** Xtensa single/dual-core, Wi-Fi + Bluetooth, 34 GPIO pins.
- **ESP32-S2:** Single-core, **Wi-Fi ONLY** (No Bluetooth).
- **ESP32-S3:** Dual-core, Includes instructions to accelerate Machine Learning (AI) apps, Bluetooth 5.
- **ESP32-C3:** Single-core RISC-V CPU, Wi-Fi + Bluetooth.

5.4 Raspberry Pi

Not a microcontroller, but a complete Single-Board Computer (SBC).

- Runs a Linux-based OS (Raspberry Pi OS).
- Features an ARM-compatible CPU, HDMI ports, USB ports, and Gigabit Ethernet.
- **Pi 4 Model B:** Quad-core 1.5GHz, up to 8GB RAM, Dual micro HDMI (supports 4K video).
- **Pi 5:** 2.4GHz CPU (Twice as powerful as Pi4), Dedicated Graphics chip, PCIe interface, Real-Time Clock (RTC).

5.5 Other SBCs

- **BeagleBone Black:** ARM Cortex-A8, 512MB RAM, HDMI. Uses **"Capes"** (Expansion boards) similar to Arduino Shields. No Wi-Fi/Bluetooth, but has Ethernet.
- **pcDuino & Cubieboard:** Linux-based ARM boards for educational/embedded purposes.

Exam Focus

Compare Raspberry Pi and ESP32 based on their Operating System, Processing Power, and Use Cases.
Ans: Pi uses a full OS (Linux), has high processing power, and is used for heavy computing/media. ESP32 uses RTOS or bare-metal, has lower power but built-in Wi-Fi/BT, and is highly affordable for basic IoT sensor nodes.

6 Lecture 7N, 10 & 12: Communication in IoT

6.1 Communications Criteria & Range

- **Short Range ($<$ 10 meter):** Bluetooth (IEEE 802.15.1), Serial cable. (Not highly scalable).
- **Medium Range (10s to 100s meters):** Wi-Fi (IEEE 802.11), Zigbee/Thread (IEEE 802.15.4), Ethernet. Used in smart homes and factories.
- **Long Range ($>$ 1 mile):** Cellular (2G/3G/4G/5G) and LPWA technologies (LoRaWAN, Sigfox, NB-IoT).

6.2 IoT Network Topologies

- **Star Topology:** All devices connect to a central base station (e.g., Cellular, Wi-Fi router). *Pros:* Easy to manage, saves battery. *Cons:* Single point of failure (if hub fails, network is down).
- **Peer-to-Peer (P2P):** Devices talk directly to each other (e.g., Bluetooth). Do not need a central hub.
- **Mesh Topology:** Devices relay data through neighboring nodes (e.g., Zigbee). *Pros:* Highly reliable and self-healing. *Cons:* High complexity and consumes more battery (nodes must stay awake to route traffic).

Exam Focus

Compare Star and Mesh Topology. Which one drains battery faster and why?
Ans: Mesh drains battery faster because nodes constantly act as routers to relay other nodes' data.

6.3 Frequency Bands

- **Licensed Spectrum:** Paid, exclusive, secure, and free from interference. Example: Cellular (NB-IoT, 4G, 5G).
- **Unlicensed Spectrum (ISM Bands):** Free for public use (Industrial, Scientific, and Medical). Includes 2.4 GHz, 5 GHz (Wi-Fi, Bluetooth) and Sub-GHz bands (433/868/915 MHz used by LoRaWAN/Sigfox). Prone to interference.

Exam Focus

What are ISM bands? Differentiate between Licensed and Unlicensed spectrum.

6.4 Constrained Devices (RFC 7228)

A standard by IETF categorizing IoT nodes with severely limited power, memory, and processing.

Class	Characteristics
Class 0 (C0)	$<$ 10 KiB RAM. Cannot run a full internet (IP) stack. Requires a proxy.
Class 1 (C1)	$\sim$ 10 KiB RAM. Can run lightweight protocols like CoAP.
Class 2 (C2)	$\sim$ 50 KiB RAM. Capable of running a full IP stack (TCP/UDP).

Memory Trick

"Class Zero – Internet Hero nahi ban sakta (No IP). Class Two – Sabkuch Do (Full IP)."

Exam Focus

Define Constrained Devices according to RFC 7228 and explain its 3 classes.

5.5 Wi-Fi & Li-Fi

- **Wi-Fi Generations:** 802.11b (Legacy), 802.11ac (Wi-Fi 5), 802.11ax (Wi-Fi 6), 802.11be (Wi-Fi 7).
- **Li-Fi (Light Fidelity):** Uses LED light bulb flickering for **Visible Light Communication (VLC)** instead of radio waves. Can reach speeds up to 224 GB/s. Extremely secure because light cannot penetrate walls (neighbors cannot steal your internet).

5.6 Bluetooth

- Forms a **Piconet** architecture (1 Master and multiple Slaves).
- **Connection Phases:**

1. **Inquiry:** Searching for nearby devices.
2. **Paging:** Establishing the connection.

- Bluetooth packet contains: Access Code, Header, and Payload.

Memory Trick

Bluetooth's "I-P" trick:
I = Inquiry (khojna/search)
P = Paging (connect karna)

5.7 LPWAN (Low Power Wide Area Networks)

The core backbone of long-range IoT. Philosophy: "Consume less battery, travel long distances".

5.7.1 1. LoRaWAN

- Uses **Spreading Factor (SF)** ranging from SF7 to SF12.
- **SF7:** High speed, but short range.
- **SF12:** Very slow speed, but maximum range.
- MAC layer developed by the LoRa Alliance. Uses unlicensed spectrum.

5.7.2 2. DASH7

- Uses **"Wake-on-Radio"**. Device sleeps, wakes up only when gateway pings it, sends data, and sleeps again. Faster than LoRa.

5.7.3 3. Sigfox

- French LPWAN technology. Highly strict on power.
- A device is limited to **140 messages per day**, and max payload is **12 Bytes**.

5.7.4 4. NB-IoT (NarrowBand IoT)

- Uses Cellular towers (Licensed band). No separate gateway needed.
- Operates in the "Guard Band" of LTE.
- High scalability: One cell tower can connect **1,00,000** NB-IoT devices.

Exam Focus

Compare LoRaWAN, Sigfox, and NB-IoT based on spectrum used, architecture, and payload capacity.

Memory Trick

LoRaWAN (Rubber Band): SF7 (kam kheencha = fast/short), SF12 (poora kheencha = slow/long).
Sigfox (Strict Teacher): Only 140 messages of 12 bytes!
NB-IoT (Jio Tower): Direct mobile tower connection, no gateway.

5.8 RFID, NFC & Zigbee

5.8.1 RFID

Used in toll collection (FASTag) or mall security tags.

- **Passive:** No internal battery. Powers up using electromagnetic energy from the reader.
- **Active:** Has its own battery. Longer range.

5.8.2 NFC (Near Field Communication)

- Operates at 13.56 MHz.
- Very short range (**4-5 cm**). Used for "Tap to Pay".

5.8.3 Zigbee (IEEE 802.15.4)

Forms a **Mesh Network** consisting of 3 specific device roles:

1. **Coordinator:** The network creator/boss. Only **one** per network.
2. **Router:** Relays/routes data between devices.
3. **End Device:** Only talks to its parent. Spends most time sleeping to save battery. Cannot route data for others.

Memory Trick

C-R-E Corporate Trick:
Coordinator = CEO (Only one).
Router = Manager (Passes data).
End Device = Employee (Does its job and sleeps).

5.9 SBC Wiring Protocols (UART, I2C, SPI)

How sensors physically connect to microcontrollers.

- **GPIO:** General pins for simple On/Off (e.g., LED).
- **UART:** Asynchronous (no clock). **2 Wires:** Tx (Transmit) and Rx (Receive).
- **I2C:** Synchronous. **2 Wires:** SDA (Serial Data) and SCL (Serial Clock). Supports multi-master/multi-slave.

- **SPI (Serial Peripheral Interface):** Synchronous. High speed (low-latency). **4 Wires:**
  - **MOSI:** Master Out Slave In.
  - **MISO:** Master In Slave Out.
  - **SCK:** Serial Clock.
  - **SS/CS:** Slave Select (to choose which slave to talk to).

Exam Focus

Explain the 4 pins of the SPI communication protocol. Which is faster, SPI or I2C?
Ans: SPI is faster.

Memory Trick

UART: "**U** Are **Two**" ⇒ 2 Wires (Tx, Rx).
I2C: "**I-Two-C**" ⇒ 2 Wires (SDA, SCL).
SPI: "**S**uper **P**ower **I**nterface" ⇒ Fastest! Needs 4 Wires.

6 Lecture 13: Network Layer & 6LoWPAN

6.1 IP Adaptation (6LoWPAN)

Definition

6LoWPAN: IPv6 over Low-power Wireless Personal Area Networks.

The Problem: IPv6 header is **40 Bytes** long. This is too large and heavy for constrained, battery-powered IoT devices (like Zigbee sensors).

The Solution: 6LoWPAN uses **Header Compression** techniques to shrink the 40-byte IPv6 header down to just **2 to 7 Bytes**.

- It supports both Star and Mesh topologies.
- **Architecture:** Contains Sensor Nodes, Routers, and an **Edge Gateway**.
- **Gateway Role:** Receives compressed 6LoWPAN packets from sensors, *de-compresses* them back into full 40-byte IPv6 packets, and forwards them to the external Cloud/Internet.

Exam Focus

What is 6LoWPAN and why is header compression necessary in IoT networks?

Memory Trick

6-Lo-W-PAN ⇒ "6 (IPv6) Low (Chhota)". Shrinking a 40kg bag into a 2kg bag so small sensors can carry it.

7 Lecture 18 & 19: Application Protocols for IoT

7.1 Transport Layer Basics: TCP vs UDP

- **TCP (Transmission Control Protocol):** Connection-oriented, reliable. Performs a 3-way handshake. (Like a strict postman who requires a signature).
- **UDP (User Datagram Protocol):** Connectionless, unreliable, lightweight. (Like a paperboy throwing the newspaper). IoT sensors predominantly use UDP to save battery.

7.2 Why HTTP is NOT suitable for IoT

- HTTP uses TCP (Port 80) and is designed for human-readable web documents.
- It is **Connection Less:** Drops connection after every request, requiring a new handshake each time.
- It is **Stateless:** The server does not remember past requests (no memory).
- The text-based headers are too heavy for constrained IoT nodes with low battery and memory.

7.3 CoAP (Constrained Application Protocol)

A specialized web transfer protocol designed for constrained nodes and constrained networks (like 6LoWPAN).

- Runs on **UDP** (Port 5683).
- Uses **DTLS** (Datagram Transport Layer Security) for security (Port 5684).
- **Header Size:** Very lightweight, only **4 Bytes**.

7.3.1 The 4 Message Types of CoAP

Because UDP is unreliable, CoAP implements its own reliability mechanism using 4 message types:

1. **Confirmable (CON):** Reliable message. The sender strictly expects an ACK in return.
2. **Non-confirmable (NON):** Unreliable message. "Fire and forget" (no ACK expected).
3. **Acknowledgement (ACK):** The confirmation reply sent in response to a CON message.
4. **Reset (RST):** Negative reply. Sent when the server receives a CON message but cannot process it or it's malformed.

7.3.2 CoAP Header/Message Format

Field	Size	Description
Version (Ver)	2 bits	CoAP version (01 for current standard).
Type (T)	2 bits	Message type: 00 (CON), 01 (NON), 10 (ACK), 11 (RST).
Token Length (TKL)	4 bits	Length of Token field (0–8 bytes).
Code	8 bits	Request method (e.g., 0.01=GET) or response status.
Message ID	16 bits	Unique identifier to match ACKs and avoid duplicates.

7.3.3 Message Exchange Models (Responses)

1. **Piggybacked Response:** If the server has the answer immediately, it sends the ACK and the Response Data together in a single message.
2. **Separate Response:** If the server needs time to process, it first sends an empty ACK. Later, it sends the Response Data in a separate new CON message.

Memory Trick

Piggybacked: Friend is ready at the door, takes the notebook, and gives you the money instantly.
Separate: Friend opens the door (ACK), goes inside to search, and returns after 10 mins with the notebook (Response).

7.3.4 Methods and Response Codes

Follows RESTful architecture (like HTTP).

- **Methods:** GET, POST, PUT, DELETE.
- **Response Codes:** 2.xx (Success - e.g., 2.01 Created), 4.xx (Client Error - e.g., 4.04 Not Found), 5.xx (Server Error - e.g., 5.00 Internal Server Error).

Exam Focus

1. Why is HTTP not suitable for constrained IoT devices?
2. Explain the 4 message types of CoAP.
3. Differentiate between Piggybacked and Separate response in CoAP.

6.4 MQTT (Message Queuing Telemetry Transport)

Created in 1999 by IBM and Eurotech to monitor oil pipelines over satellite networks.

- Runs on **TCP** (Port 1883). Secure version runs on Port 8883/8884.
- Extremely lightweight: Header is only **2 Bytes** minimum.

6.4.1 Architecture

Operates on a **Publish/Subscribe** model.

- **Publisher:** The sensor/device that generates data and sends it to the broker.
- **Broker:** The central server/manager (e.g., Eclipse Mosquitto) that receives all data and distributes it to interested subscribers.
- **Subscriber:** The client (e.g., mobile app) that registers interest in receiving specific data.

Note: The Publisher and Subscriber never connect directly; everything routes through the Broker.

6.4.2 Topics and Wildcards

Data is routed using **Topics** (a case-sensitive, hierarchical string like a URL path, e.g., `home/bedroom/temperature`).
Subscribers can use **Wildcards** to subscribe to multiple topics at once:

- + (Single Level Wildcard): Replaces exactly one level. (e.g., `home/+/temperature`).
- # (Multi-Level Wildcard): Replaces all subsequent levels. Must be at the end. (e.g., `home/#`).
- **$** (System Topic): Used for internal broker statistics (e.g., `$SYS/`). Cannot be matched by + or #.

6.4.3 Quality of Service (QoS) Levels

MQTT guarantees message delivery using 3 QoS levels:

1. **QoS 0 ("At most once"):** "Fire and forget". The publisher sends the message once. No ACK is expected. Fast but unreliable.
2. **QoS 1 ("At least once"):** Guarantees delivery, but duplicate messages may occur if the **PUBACK** (Acknowledgement) is lost and the message is retransmitted.
3. **QoS 2 ("Exactly once"):** The safest and highest level. Guarantees no message loss and NO duplicates. Requires a **4-step handshake**:
   - **PUBLISH:** Sender sends data and stores a copy.
   - **PUBREC:** Broker receives, stores a copy, and replies.
   - **PUBREL:** Sender receives PUBREC, deletes its copy, and tells Broker to release.
   - **PUBCOMP:** Broker completes the process and sends final confirmation.

Exam Focus

1. Explain the architecture of MQTT.
2. What are the 3 levels of QoS in MQTT? Differentiate between QoS 1 and QoS 2.

Memory Trick

QoS 0: Normal postcard (No receipt).
QoS 1: Speed post (Signature required, but might get duplicate delivery attempt if signature lost).
QoS 2: Bank OTP + ATM Card Delivery. Hand-to-hand, verified 4-step process to ensure zero duplicates.

6.4.4 MQTT PUBLISH Packet Structure

The packet has three parts:

1. **Fixed Header (2 bytes):** Contains the Packet Type (0011 for PUBLISH) and 4 Control Flags:
   - **DUP:** Duplicate flag.
   - **QoS:** Level (0, 1, or 2).
   - **RETAIN:** Should the broker save this message for future subscribers?
2. **Variable Header:** Contains the Topic Name and a Packet Identifier (if QoS > 0).
3. **Payload:** The actual data being sent.

7 Lecture 21–34: Cloud Computing & Virtualization

7.1 HPC vs HTC

- **HPC (High Performance Computing):** Supercomputers. Focuses on raw computational speed (Teraflops/Petaflops) for complex tasks. Used by scientists.
- **HTC (High Throughput Computing):** Cloud servers. Focuses on handling millions of concurrent, smaller tasks (high throughput) simultaneously (e.g., Google search queries).

7.2 Cloud Service Models (Pizza as a Service)

- **On-Premises:** You manage everything (Hardware, OS, App). Like making pizza at home.
- **IaaS (Infrastructure as a Service):** Cloud provides bare-metal servers/storage. You install the OS and App (e.g., AWS EC2). Like buying a frozen pizza and baking it at home.
- **PaaS (Platform as a Service):** Cloud provides the platform (OS, runtime). You just write the code (e.g., Google App Engine). Like pizza delivery.
- **SaaS (Software as a Service):** Cloud manages everything. You just use the application (e.g., Gmail, Netflix). Like eating at a restaurant.

Memory Trick

IaaS = **I** am a Server.
PaaS = **P**latform all Set.
SaaS = **S**oftware already Served.

7.3 Virtualization

Splitting a physical computer into multiple isolated Virtual Machines (VMs) using software.

- **Hypervisor:** The software that enables virtualization.
  - **Type 1 (Bare-metal):** Installed directly on hardware. Fastest. (e.g., VMware ESXi).
  - **Type 2 (Hosted):** Installed as an application on an existing Host OS. (e.g., VirtualBox).
- **Containerization (e.g., Docker):** Unlike VMs which require a full, heavy Guest OS, containers share the Host OS kernel. They only package the app and its dependencies, making them extremely lightweight (MBs vs GBs) and fast.

Exam Focus

Differentiate between Type 1 and Type 2 Hypervisor. Compare Virtual Machines with Containers.

7.4 AWS Storage Computing

- **Amazon S3:** Standard fast cloud storage (like Google Drive).
- **Amazon Glacier:** Cold storage for backups and archiving. Very cheap, but data retrieval takes hours.
- **AWS Lambda: Serverless computing.** You upload code, AWS runs it automatically when triggered. No need to manage servers.

7.5 Data Centers & CAP Theorem

- **Google File System (GFS) / Hadoop (HDFS):** Large files are broken into **64 MB Chunks**. These chunks are replicated across multiple servers for fault tolerance.

Definition

CAP Theorem: In any distributed data store, you can only simultaneously guarantee TWO out of the following three properties:

1. **C - Consistency:** Every read receives the most recent write or an error.
2. **A - Availability:** Every request receives a non-error response.
3. **P - Partition Tolerance:** The system continues to operate despite an arbitrary number of messages being dropped by the network.

Exam Focus

Explain the CAP theorem in Cloud Storage.

6.6 MQTT Control Packets in Detail

There are **14 Control Packets** in MQTT. (Values 0 and 15 are forbidden).

6.6.1 1. Connection Packets

- **CONNECT (1):** Client requests connection to Broker.
- **CONNACK (2):** Broker acknowledges the connection.
- **DISCONNECT (14):** Client gracefully disconnects.
- **PINGREQ (12) / PINGRESP (13):** Keep-alive mechanism. Client sends PINGREQ to check if connection is alive; Broker replies with PINGRESP.

Keep-Alive Timer: Set during the CONNECT phase. If the broker does not receive a PINGREQ within **1.5** $\times$ **Keep-Alive Time**, it assumes the client is dead and drops the connection.

6.6.2 2. Publishing Packets (QoS Mechanisms)

- **PUBLISH (3):** Sends the actual data.
- **PUBACK (4):** Acknowledgement for QoS 1.
- **PUBREC (5), PUBREL (6), PUBCOMP (7):** Used in the 4-step handshake for QoS 2.

6.6.3 3. Subscription Packets

- **SUBSCRIBE (8):** Client requests to listen to topics. Payload contains a list of topics and their desired QoS levels.
- **SUBACK (9):** Broker confirms subscription and grants QoS levels.
- **UNSUBSCRIBE (10) / UNSUBACK (11):** Client stops listening to a topic.

6.7 Deep Dive: The CONNECT Packet

The CONNECT packet acts as a "Login Form" for the client.

- **Fixed Header:** Type=1 (CONNECT), Flags=0000.
- **Variable Header:** Contains Protocol Name ("MQTT"), Version, Keep-Alive Timer, and **Connect Flags**.
- **Payload:** Contains Client Identifier (Mandatory), Username, Password, Will Topic, and Will Message.

CONNECT Flags (8 bits in Variable Header):

Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
User-name	Pass-word	Will Retain	Will QoS (Bit 1)	Will QoS (Bit 0)	Will Flag	Clean Session	Reser-ved (0)

Definition

Last Will and Testament (Will Topic/Message): An advanced feature. If a client disconnects unexpectedly (e.g., battery dies) without sending a DISCONNECT packet, the Broker automatically publishes the "Will Message" to all subscribers to notify them that the device is offline.

6.8 MQTT vs HTTP (Crucial Comparison)

MQTT is highly optimized for IoT, whereas HTTP is designed for human web browsing.

Feature	**HTTP**	**MQTT**
Design Intent	Document-based (HTML, heavy)	Data-based (Sensor values, lightweight)
Device Type	Not optimized for constrained devices	Optimized for resource-constrained devices
Throughput	Low (Needs constant TCP handshakes)	High (Persistent connection)
Reliability	No QoS built-in	Strict QoS (Levels 0, 1, 2)
Overhead	Lengthy, heavy headers	Extremely short fixed header (min 2 Bytes)
Performance	High bandwidth consumption	**20x faster, 50x less network traffic**

Exam Focus

1. Explain the role of the Keep-Alive timer and PINGREQ/PINGRESP in MQTT.
2. Compare MQTT and HTTP comprehensively for IoT applications.
3. What is the purpose of the "Last Will and Testament" feature in the CONNECT packet?

6.9 MQTT Return Codes

6.9.1 CONNACK Return Codes

- **0x00:** Success (Connection Accepted)
- **0x01:** Unacceptable Protocol Version
- **0x02:** Identifier Rejected
- **0x03:** Server Unavailable
- **0x04:** Bad Username/Password
- **0x05:** Not Authorized

6.9.2 SUBACK Return Codes

- **0x00:** Success (QoS 0 granted)
- **0x01:** Success (QoS 1 granted)
- **0x02:** Success (QoS 2 granted)
- **0x80:** Failure

Note: UNSUBACK, PUBACK, PUBREC, PUBCOMP do not contain return codes.

6.10 CoAP vs MQTT (Crucial Comparison)

This is a high-probability exam topic comparing the two main IoT application protocols.

Parameters	CoAP	MQTT
Transport Layer	UDP	TCP
Architecture Model	Request-Response Model	Publish-Subscribe Model
Communication Mode	One-to-One	Many-to-Many
Header Size	4 Bytes	2 Bytes
Message Labelling	2-Byte Message ID	Not available
Port No	5683 (Secure: 5684)	1883 (Secure: 8883)
Reliable Message	Confirmable, Non-Confirmable	QoS 0, 1, 2

Exam Focus

Explain the difference between CoAP and MQTT in terms of Transport Layer, Architecture Model, and Reliable Messaging.

7 Lecture 21–33: Cloud Computing & Data Centers

7.1 History of Computing

- **Abacus (4th Century BC):** First calculating device (Babylonia).
- **Pascaline (1642):** Adding machine.
- **Analytical Engine (1837):** Built by Charles Babbage. Introduced the concept of Memory and ALU (first general-purpose computer design).
- **Punch Card (1884):** Herman Hollerith's system for storing data using holes.

7.2 Cloud Computing as a Utility

Prof. Leonard Kleinrock (1965) and Prof. Rajkumar Buyya (2009) envisioned computing becoming a public utility (the "5th Utility" after water, electricity, gas, and telephone). Cloud Computing follows a **"Pay-as-you-go"** model where resources are accessed over the internet without managing physical hardware.

7.3 HPC vs HTC

- **HPC (High Performance Computing):** Supercomputers focusing on "Raw Speed" (TFlops/PFlops) for complex scientific calculations.
- **HTC (High Throughput Computing):** Cloud servers focusing on handling millions of smaller, concurrent tasks (High Throughput).

7.4 Cloud Service Models (Pizza as a Service)

1. **On-Premises:** Manage everything yourself (Hardware, OS, App). Like making a pizza from scratch at home.
2. **IaaS (Infrastructure as a Service):** Cloud provider gives bare-metal servers/storage (e.g., AWS EC2). You manage OS and App. Like a take-and-bake pizza.
3. **PaaS (Platform as a Service):** Cloud provider manages servers and OS. You just upload your code (e.g., Google App Engine). Like pizza delivery.
4. **SaaS (Software as a Service):** Cloud provider manages everything. You just use the software (e.g., Gmail, Google Drive). Like eating at a restaurant.

7.5 Data Centers & Distributed Systems

- **Power Consumption:** Data centers consume roughly 0.5% of global electricity.
- **Google File System (GFS) / Hadoop (HDFS):** Large files are divided into fixed-size **64 MB Chunks**. A Master Node tracks where chunks are stored on Chunk Servers. To ensure fault tolerance, chunks undergo **Replication** (multiple copies are saved across different servers).

7.6 CAP Theorem

A fundamental rule in distributed cloud databases. You can only simultaneously achieve **TWO** out of the three properties:

1. **Consistency (C):** All nodes see the same data at the same time.
2. **Availability (A):** Every request receives a response (system is always online).
3. **Partition Tolerance (P):** The system continues to operate despite network failures/drops.

Since network partitions (P) are unavoidable in distributed systems, architects must choose between prioritizing Consistency (CP) or Availability (AP).

Memory Trick

CAP Theorem Trick: "Choose Any Two!" You cannot have 100% C, A, and P simultaneously.

7.7 Disadvantages of Cloud Computing

- Requires continuous internet connectivity.
- Security and privacy concerns (data stored on third-party servers).
- **Vendor Lock-in:** Difficulty and cost associated with moving data/services from one provider (e.g., AWS) to another.

8 Lecture Series: Cloud Computing In-Depth

8.1 Evolution of Computing

- **Analytical Engine (1837):** Designed by Charles Babbage. Considered the first design for a **general-purpose computer** because it introduced the ALU (Arithmetic Logic Unit), integrated memory, and control flow (conditional branching and loops).
- **Mark I (1937–1944):** Built by Howard Aiken with IBM funding. An electromechanical giant spanning 51 feet in length with 500 meters of wiring.
- **John-Berry Computer / ABC (1942):** The **first electronic digital computer**. Shifted from mechanical parts to **Vacuum Tubes** (300 tubes) and introduced the **Binary System (0s and 1s)** which is still used today.
- **IBM System/360 (1964):** The first "family of computers" designed to handle both commercial (business) and scientific applications simultaneously.

8.2 HPC vs HTC

HPC (High-Performance Computing)	HTC (High-Throughput Computing)
Goal: Raw Speed for solving a single, massive problem.	**Goal:** Maximum Volume. Executing millions of smaller tasks concurrently.
Metrics: Gflops, Tflops, Pflops.	**Metrics:** Number of tasks completed per unit time.
Use Case: Climate modelling, computational biology.	**Use Case:** Web search, e-commerce, cloud web hosting.
Hardware: Supercomputers.	**Hardware:** Cloud Data Centers.

8.3 Computing Architectures

1. **Centralized Computing:** Processors, memory, and storage are fully shared. Tightly coupled within one integrated OS. (e.g., Mainframes).
2. **Parallel Computing:** A large problem is broken down and solved simultaneously by tightly or loosely coupled processors to achieve speed.
3. **Distributed Computing:** Multiple autonomous computers (each with private memory) communicate over a network via **message passing**. This is the foundation of modern Cloud systems.

*Note: The union of Parallel and Distributed computing is often called **Concurrent Computing**.*

8.4 National Supercomputing Mission (NSM)

An initiative by the Government of India to empower R&D by deploying high-end supercomputing grids.

- **PARAM Series:** Indigenous supercomputers built by C-DAC.
- **Examples: PARAM Pravega** (IISc Bangalore, ∼14,400 cores), **PARAM Seva**, **PARAM Smriti**. Other massive clusters are deployed at IIT Kanpur, IIT Gandhinagar, and IISER Pune.

8.5 The "Utility" Vision & NIST Definition

Definition

NIST Definition of Cloud Computing: A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or service provider interaction.

5 Essential Characteristics (NIST):

1. **On-demand self-service:** Provision resources without human interaction.

2. **Broad network access:** Accessible from any standard device over the internet.
3. **Resource pooling:** Provider's hardware is shared dynamically among multiple users (multi-tenant model).
4. **Rapid elasticity:** Automatically scale up or down based on sudden traffic spikes.
5. **Measured service:** Pay-as-you-go billing model.

Exam Focus

Briefly trace the modern history of computing, from the Analytical Engine to Cloud Utility computing. Explain the 5 essential characteristics defined by NIST.

7.6 Cloud Service and Deployment Models

7.6.1 Service Models (XaaS)

- **IaaS (Infrastructure as a Service):** Provider supplies bare-metal hardware (servers, network). User manages OS, runtime, and applications. (e.g., AWS EC2).
- **PaaS (Platform as a Service):** Provider manages hardware and OS. User focuses entirely on deploying code/applications. (e.g., AWS Elastic Beanstalk).
- **SaaS (Software as a Service):** Fully ready-made software running on the internet. (e.g., Gmail, Google Docs).
- **XaaS (Everything as a Service):** An umbrella term indicating that almost any IT function can be delivered as a cloud service.

7.6.2 Deployment Models

- **Public Cloud:** Open to the general public. Cost-effective and highly scalable (e.g., AWS, Azure).
- **Private Cloud:** Exclusively used by a single organization. Highly secure.
- **Hybrid Cloud:** A combination of Public and Private clouds. Sensitive data stays private, while regular workloads run on the public cloud.
- **Community Cloud:** Shared by multiple organizations with common goals (e.g., top research institutes sharing an HPC grid).

7.7 Shared Responsibility Model

Security in the cloud is a shared responsibility.

- **Always Provider's Responsibility:** Physical data center security, physical network, and physical hosts.
- **Always Customer's Responsibility:** Actual data/information, account passwords, identities, and endpoint devices.
- **Shifting Responsibility:** In IaaS, OS patching is the customer's job. In PaaS/SaaS, OS management shifts to the provider.

7.8 Elasticity vs. Scalability

Elasticity	Scalability
Ability to automatically expand or shrink resources based on **sudden** workload changes.	Persistent addition of resources to handle **static/planned** workload growth.
Nature: Temporary (Rubber-band effect).	**Nature:** Permanent (Building up infrastructure).

7.9 Virtualization Containerization

7.9.1 Hypervisors

A **Hypervisor (Virtual Machine Monitor)** is the software layer that allows multiple Virtual Machines to run on the same physical hardware independently.

- **Type 1 (Bare-metal):** Installed directly on physical hardware. Extremely fast. Used in enterprise data centers. (e.g., VMware ESXi, Xen).
- **Type 2 (Hosted):** Installed as a software application on top of an existing Host OS (Windows/Mac). Slower due to overhead. (e.g., Oracle VirtualBox).

7.9.2 Full vs. Para-Virtualization

- **Full Virtualization:** The Guest OS is unaware it is virtualized. The hypervisor intercepts hardware requests via **trap-and-emulate**. Slower but highly compatible.
- **Para-Virtualization:** The Guest OS knows it is a VM. It uses **Hypercalls** to communicate directly with the hypervisor, resulting in near-native (raw) performance. (e.g., Xen).

7.9.3 Containers vs. VMs

- **Virtual Machines:** Heavyweight. Each VM contains a full Guest OS. Takes GBs of space and minutes to boot. High hardware-level isolation.
- **Containers (e.g., Docker):** Lightweight. Contains only the application and libraries, sharing the Host OS Kernel. Takes MBs of space and boots in milliseconds. Managed at scale by **Kubernetes**.

Exam Focus

1. Differentiate between Elasticity and Scalability with examples.
2. Explain the Shared Responsibility Model in Cloud Computing.
3. Differentiate between Type 1 and Type 2 Hypervisors.
4. Compare Virtual Machines with Containerization.

7.10 Major Cloud Providers & AWS Ecosystem

The global cloud market is dominated by three major providers: **AWS (31%)**, **Microsoft Azure (24%)**, and **Google Cloud Platform (GCP) (11%)**. Cloud providers ensure high reliability by organizing data centers into **Regions** containing multiple isolated **Availability Zones (AZs)**.

7.10.1 Key AWS Services

- **Amazon EC2 (Elastic Compute Cloud):** IaaS service providing secure, resizable compute capacity (Virtual Machines) for hosting complete operating systems.
- **AWS Lambda:** Serverless computing service. Users run code (Python, Java) without provisioning servers; AWS executes it dynamically based on events/triggers.
- **Amazon RDS (Relational Database Service):** Managed SQL databases (MySQL, Oracle). Automates backups and patching.
- **Amazon DynamoDB:** Fully managed NoSQL database providing single-digit millisecond latency at massive scale.
- **Amazon S3 (Simple Storage Service):** Object storage for general-purpose files, media, and backups. Highly scalable.
- **Amazon Glacier:** Low-cost "Cold Storage" for long-term archiving. Retrieval can take minutes to hours.
- **AWS WAF (Web Application Firewall):** Protects web apps from common exploits (like SQL injection) and bots.
- **AWS Management Console:** A unified web-based dashboard to access and control all AWS resources.

7.11 Data Center Architecture & Networking

Data centers consume massive power (approx. 0.5% of global electricity). Engineers focus on **Green Computing** to reduce energy footprints.

7.11.1 Structural Hierarchy

- **Server:** Contains memory (DRAM) and disk drives.
- **Rack:** A group of servers (e.g., 40 servers) connected via a **Rack-Level Switch**.
- **Cluster:** Multiple racks connected via **Cluster-Level Switches**.

7.11.2 Cooling Systems (Raised Floors)

Massive servers produce immense heat. Data centers use **Raised Floors** (a hollow gap under the floor tiles). Cold air is pumped under the floor and escapes through perforated tiles directly in front of the server racks (**Cold Aisle**). The servers expel hot air from the back (**Hot Aisle**) towards ceiling AC returns.

7.11.3 Interconnection Networks & Fat-Tree Topology

Modern data centers are **Switch-Centric** to avoid modifying OS routing on servers.

- **Fault Tolerance:** Networks employ multiple redundant paths between any two servers to avoid a Single Point of Failure.
- **Fat-Tree Topology:** A three-layer design (Core, Aggregation, Edge). The links become "fatter" (higher bandwidth) towards the root/core to prevent network bottlenecks.
- **Pod:** A functional grouping containing edge switches, aggregation switches, and connected servers.

7.11.4 Modular Data Centers (e.g., SGI ICE Cube)

Instead of building a traditional brick-and-mortar facility, **Modular Data Centers** pack tens of thousands of compute cores and petabytes of storage inside a standard truck-towed shipping container.

- **Motivation:** High mobility, lower electricity/cooling costs, and the ability to drop immense compute power in remote areas (e.g., rural healthcare clinics).

7.12 Cloud Storage Models

Understanding how data is stored on remote servers is critical for system design.

7.12.1 1. Block Storage (e.g., Amazon EBS)

- **Concept:** Operates like a raw, unformatted physical hard drive attached to a VM. Data is stored in fixed-size blocks.
- **Performance:** Extremely fast read/write speeds because there is no metadata/folder hierarchy traversal overhead.
- **Use Case:** Ideal for intensive workloads like Relational Databases (OS sees it as a disk).

7.12.2 2. File Storage (e.g., Amazon EFS)

- **Concept:** Operates like a shared network folder with a hierarchical tree structure (folders inside folders).
- **Performance:** Slower than Block Storage due to **Metadata Traversal** (the system must read index files at each folder level before reaching the actual data).
- **Use Case:** Ideal when multiple VMs/servers need simultaneous read/write access to shared configuration or application files.

7.12.3 3. Object Storage (e.g., Amazon S3)

- **Concept:** A "flat" storage structure. Data is bundled as an **Object** containing the file data, metadata, and a unique ID (Key).
- **How it Works:** Uses **O(1) Hash Lookups**. The cloud maintains a massive Master Hash Table mapping the unique ID directly to the physical drive sector. When a file is requested via its HTTP URL, the system jumps directly to the location without traversing folders.
- **Use Case:** Infinitely scalable. Perfect for unstructured data like media files (images, videos), backups, and web assets.

Exam Focus

Compare Object, Block, and File Storage. Why is Object Storage considered infinitely scalable?

7.13 CAP Theorem in Cloud Databases

A fundamental rule for distributed data stores. It states that a distributed system can only simultaneously guarantee **TWO** out of the following three properties:

1. **C - Consistency:** Every read receives the most recent write (all nodes show identical data simultaneously).
2. **A - Availability:** Every request receives a (non-error) response, even if some nodes are failing.
3. **P - Partition Tolerance:** The system continues to operate despite network failures/drops between nodes.

Reality Check: Since network partitions (P) are unavoidable, architects must choose between **CP** (Consistency + Partition Tolerance) or **AP** (Availability + Partition Tolerance).

7.14 Service Level Agreement (SLA)

A formal, legal contract between a Cloud Service Provider (SP) and a Service Consumer (SC).

- **SLA (Service Level Agreement):** The master document outlining services, metrics, auditing mechanisms, and the penalty/remedy model.
- **SLO (Service Level Objective):** The specific, objectively measurable promises within the SLA (e.g., "We promise 99.9% uptime").
- **SLI (Service Level Indicator):** The actual real-time measurement of the service (e.g., dashboard showing 99.95% uptime).

7.14.1 Types of SLAs

1. **Off-the-shelf (Non-negotiable) SLA:** Standard "Take it or leave it" terms provided to general users via an online portal. Not ideal for highly mission-critical data.

2. **Negotiable SLA:** Custom terms negotiated between enterprise consumers (e.g., a large bank) and the provider, usually via legal agents.

9 Real-World IIoT System Calculations & Engineering

9.1 PIR Sensor Battery Life Calculation

A smart sensor operates in two states: **Active (Wake mode)** and **Idle (Sleep mode)**.
Given Data:

- Battery Capacity: 1500 mAh at 5V
- Idle Current: 0.02 mA
- Active Current: 0.5 mA
- Motion Detection: 30 times per hour, staying active for 2 seconds each time.

Step-by-Step Solution:

1. **Time Distribution (per hour):**
   - Active Time $= 30 \times 2 = 60$ sec/hr $= \frac{60}{3600}$ hr ≈ 0.0167 hr.
   - Idle Time $= 3600 - 60 = 3540$ sec/hr $= \frac{3540}{3600}$ hr ≈ 0.983 hr.
2. **Energy Consumption (per 24 hours):** Formula: $Energy = \text{Voltage} \times \text{Current} \times \text{Time}$
   - Active Energy (1 Day) $= 5 \times 0.5 \times 0.0167 \times 24 \approx 1.002$ mWh.
   - Idle Energy (1 Day) $= 5 \times 0.02 \times 0.983 \times 24 \approx 2.359$ mWh.
   - Total Daily Energy $= 1.002 + 2.359 = 3.36$ mWh.
3. **Battery Life:**
   - Total Battery Energy $= 1500$ mAh $\times$ 5V $= 7500$ mWh.
   - Life in Days $= \frac{\text{Total Capacity}}{\text{Daily Consumption}} = \frac{7500}{3.36} \approx$ **2232 Days**.

9.2 Sensor Coverage Area Calculation

A ceiling-mounted sensor projects a detection cone, covering a circular area on the floor.
Given Data: Ceiling Height (h) = 2.5m (Case 1), Detection Angle = 120°.

Step-by-Step Solution:

1. **Radius of Coverage (x):** The cone forms a right-angled triangle with a half-angle of 60°.

$$\tan(60^\circ) = \frac{\text{Perpendicular (Radius } x)}{\text{Base (Height } h)} \implies \sqrt{3} = \frac{x}{2.5} \implies x = 2.5\sqrt{3} \text{ m}$$

2. **Area Covered:**

$$\text{Area} = \pi x^2 = \pi(2.5\sqrt{3})^2 = \pi \times 6.25 \times 3 \approx 18.75\pi \approx \mathbf{58.9\ m^2}$$

Note: If the height increases to 3.5m, Area $= \pi(3.5\sqrt{3})^2 \approx 115.5m^2$. *Mounting slightly higher almost doubles the coverage!*

9.3 Wireless Signal Loss: Friis Equation & FSPL

9.3.1 Friis Transmission Equation

Determines the received power (P_R) in a free-space radio link.

$$P_R = P_t \times G_t \times G_R \left(\frac{c}{4\pi r f}\right)^2$$

Where:

- P_t: Transmitter Power, P_R: Receiver Power.
- G_t, G_R: Antenna Gains (Focusing capacity).
- r: Distance (meters), f: Frequency (Hz), c: Speed of light (3×10^8 m/s).

9.3.2 Free Space Path Loss (FSPL) in dB

If we ignore antenna gains ($G_t = 1, G_R = 1$), the loss factor alone is FSPL. We convert it to Decibels (dB) because the ratio is extremely large.

$$\text{FSPL} = \left(\frac{4\pi r f}{c}\right)^2$$

Taking $10\log_{10}$ on both sides:

$$\text{FSPL(dB)} = 20\log_{10}(r) + 20\log_{10}(f) + 20\log_{10}\left(\frac{4\pi}{c}\right)$$

Since $c = 3 \times 10^8$ and $\pi = 3.14$, the constant term $20\log_{10}\left(\frac{4\pi}{c}\right) \approx -147.55$.

Definition

Master FSPL Formula:

$$\text{Loss (dB)} = 20\log_{10}(r) + 20\log_{10}(f) - 147.55$$

9.3.3 FSPL Numericals

Case 1: 2.4 **GHz at** 100**m**

$$\text{Loss} = 20\log(100) + 20\log(2.4 \times 10^9) - 147.55$$

$$\text{Loss} = 40 + [20\log(10^9) + 20\log(2.4)] - 147.55$$

$$\text{Loss} = 40 + 180 + 7.6 - 147.55 \approx \mathbf{80.05\ dB}$$

Case 2: 900 **MHz at** 100**m**

$$\text{Loss} = 20\log(100) + 20\log(9 \times 10^8) - 147.55$$

$$\text{Loss} = 40 + [20\log(10^8) + 20\log(9)] - 147.55$$

$$\text{Loss} = 40 + 160 + 19 - 147.55 \approx \mathbf{71.45\ dB}$$

Memory Trick

Rule of Thumb: Frequency (Hz) $\uparrow \implies$ Signal Loss $\uparrow$. Higher frequency waves (like 5G or 2.4GHz) exhaust energy faster over the air compared to low-frequency waves (like 900MHz or FM Radio).

8.4 Networking Protocols & LoRa Planning

8.4.1 The Hidden Terminal Problem & CSMA/CA

When Node A and Node B can communicate with Node C (Router) but are out of range of each other, they are "hidden" from each other. If both transmit simultaneously, a collision occurs at C.

Solution in Wi-Fi (CSMA/CA with RTS/CTS):

1. **RTS (Request to Send):** A asks C for permission to send.
2. **CTS (Clear to Send):** C broadcasts permission to A. Since B also hears C, B knows the channel will be busy and stays quiet.
3. A transmits data safely.

Solution in Bluetooth (Master/Slave TDMA): Bluetooth does NOT use CSMA/CA. It uses strict **Time Division Multiple Access (TDMA)**. The Phone (Master) creates a rigid timetable. Earbuds (Slave 1) and Smartwatch (Slave 2) only transmit in their predefined millisecond slots. Collision is mathematically impossible.

8.4.2 LoRa Network Planning: Fresnel Zone & Earth Bulge

When transmitting signals over long distances (e.g., 10 km), radio waves spread in a 3D elliptical shape known as the **Fresnel Zone**. For successful line-of-sight communication, this zone must not touch the Earth's surface (curvature/bulge) or obstacles.

1. Maximum Fresnel Radius (r): Formula: $r = 8.657 \times \sqrt{\frac{D}{f}}$ Where D is distance in km (10) and f is frequency in GHz (0.868 for LoRa). $r = 8.657 \times \sqrt{\frac{10}{0.868}} \approx \mathbf{29.38\ m}$.

2. Earth Bulge Height (H): The Earth's curvature pushes the ground upwards exactly at the midpoint ($\frac{\alpha}{2}$). $H = R - R\cos(\frac{\alpha}{2})$, where $R = 6371$ km (Earth's radius).

3. Required Tower Height (h): To prevent the Fresnel zone from touching the ground:

$$\textbf{Tower Height (h)} = \text{Fresnel Radius (r)} + \text{Earth Bulge (H)}$$

8.5 AODV Routing Protocol (Ad hoc On-Demand)

A reactive routing protocol that builds routes only when needed, reducing memory waste in IoT sensors.

1. **RREQ (Route Request - Broadcast):** Source node shouts to neighbors searching for the destination. Every node along the path stores **backward information** (Reverse Path) to know where the request came from.
2. **RREP (Route Reply - Unicast):** Destination receives RREQ and sends RREP back along the stored reverse path. As RREP travels back, nodes store the **forward information** (Forward Path).
3. **Data Transmission:** The source now has a fixed path and sends actual data.
4. **RERR (Route Error):** If a link breaks midway, the detecting node sends RERR backward to stop data transmission.

8.6 IP Addressing & Wi-Fi Congestion

IPv6 Structure: 128-bit address, split into 8 blocks of 16 bits. Represented in Hexadecimal (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334). **Zero Compression Rule:** Continuous blocks of zeros can be replaced by '::' (e.g., '12AB::' implies all following blocks are zero).

Wi-Fi Congestion Math:

- **Signal Speed:** Wi-Fi travels at light speed (3×10^8 m/s). Time to reach 30m is $t = \frac{30}{3\times10^8} = 100$ ns.
- **Congestion Check:** Router max capacity = 400 mbps. If there are 50 users, each needing 5 mbps, total demand = 250 mbps. Since $250 < 400$, there is **NO congestion**.

8.7 Cellular Networks & Access Technologies

- **Hexagonal Cells:** Engineers use hexagons instead of circles to provide perfect coverage without leaving gaps (dead zones) or overlapping unevenly.
- **FDMA (1G):** Users are separated by different frequency lanes. *Drawback:* If a user is idle, the entire frequency lane is wasted.
- **TDMA (2G/GSM):** Users share the same frequency but take turns in specific time slots. A preamble tells devices when to speak.
- **OFDM (4G/5G):** A grid (matrix) system combining Time (X-axis) and Frequency (Y-axis) into tiny **Resource Blocks**. Users can rent multiple blocks simultaneously, boosting speeds by 100x.